

BUYLOG · ISSUE 2

OCR 파싱 개선 & 웹 카메라 연동

Gemini 2.5 Flash 멀티모달 · getUserMedia · 조건부 플랫폼 분기



Gemini 2.5 Flash



Web Camera API



Fallback 체계



responseSchema 강제

Buylog Team — 2026

Flutter Web · Dart · Supabase

TABLE OF CONTENTS

목차

1	담당 기능 한눈에 보기 구현 범위 요약 및 관련 파일	2	전체 아키텍처 OCR 처리 흐름 · Fallback 체계
3	핵심 코드 리뷰 ①: Gemini 멀티모달 OCR 이미지 → JSON 직접 변환 · responseSchema	4	핵심 코드 리뷰 ②: Fallback 파싱 Google Vision → Gemini 텍스트 파싱 → Regex
5	핵심 코드 리뷰 ③: 웹 카메라 구현 getUserMedia · 조건부 플랫폼 분기	6	기술적 의사결정 및 트레이드오프 모델 선택 · CORS 이슈 · Fallback 설계
7	Before / After 성능 비교 파싱 정확도 · 카메라 UX 개선	8	발표 흐름 가이드 데모 순서 및 시연 포인트

1

담당 기능 한눈에 보기



Gemini 멀티모달 OCR

영수증 이미지를 Gemini 2.5 Flash에 직접 전송해 상품 목록을 JSON으로 추출. 기존 Regex 방식 대비 비정형 영수증에서도 높은 정확도 달성.

ocr_service.dart



3단계 Fallback 체계

Gemini 멀티모달 실패 시 → Google Vision 텍스트 추출 → Gemini 텍스트 파싱 → Regex 순으로 단계적 fallback을 적용해 서비스 안정성 확보.

ocr_service.dart



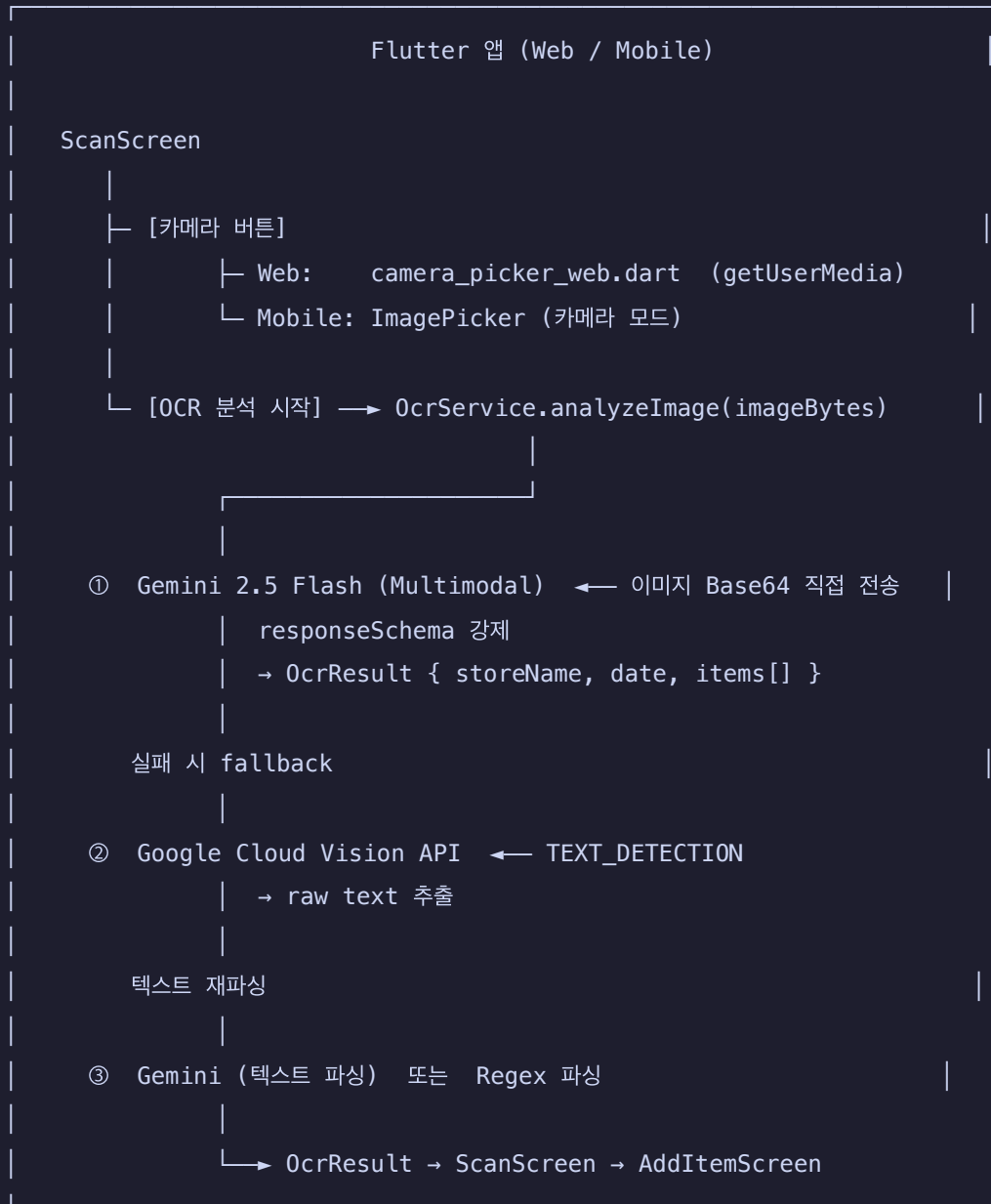
웹 카메라 직접 연동

Flutter Web에서 image_picker가 갤러리만 열리는 한계를 getUserMedia API로 해결. 실시간 카메라 미리보기 → 촬영 → OCR 처리.

camera_picker_web.dart

기능	설명	관련 파일
Gemini 멀티모달 OCR	이미지 → Base64 인코딩 → Gemini API → JSON 파싱 (단일 호출)	ocr_service.dart
responseSchema 강제	storeName / date / items 구조를 JSON Schema로 명시해 파싱 실패 방지	ocr_service.dart
유연한 날짜 파싱	2026.04.08 / 2026/04/08 / 2026년 4월 8일 등 다양한 형식 정규화	ocr_service.dart
Google Vision Fallback	Gemini API 실패 시 Vision API로 raw 텍스트 추출 후 재파싱	ocr_service.dart
플랫폼 조건부 카메라	Web: getUserMedia 오버레이 / Mobile: ImagePicker 카메라 모드	camera_picker.dart
카메라 UI 오버레이	HTML video + canvas로 풀스크린 촬영 UI 구현, 파일 선택 fallback 내장	camera_picker_web.dart

2 전체 아키텍처 — OCR 처리 흐름



카메라 입력 플랫폼 분기

```

camera_picker.dart (조건부 export)
├── dart.library.js_interop 존재 (Web 빌드)

```

```

|      └─ camera_picker_web.dart
|
|      getUserMedia({video:{facingMode:'environment'}})
|      → video 미리보기 → canvas 캡처 → Uint8List
|      (실패 시) → file input capture="environment"
|
└─ 그 외 (Mobile / Desktop)
    └─ camera_picker_stub.dart (null 반환)
        → ImagePicker().pickImage(source: ImageSource.camera)

```



왜 단일 Gemini 호출인가?

기존 방식(Vision API → 텍스트 → Regex)은 2번의 API 호출과 복잡한 패턴 매칭이 필요했습니다. Gemini 멀티모달은 이미지를 직접 이해하므로

API 호출 1회, 레이턴시 절반, 정확도 대폭 향상

이 가능합니다.

3

핵심 코드 리뷰 ① — Gemini 멀티 모달 OCR

lib/services/ocr_service.dart

3-1. responseSchema — 응답 구조 강제

Gemini가 항상 동일한 JSON 구조를 반환하도록 `responseSchema` 를 명시합니다. 이 없이는 자유 형식 텍스트가 반환돼 파싱 실패가 잦습니다.

```
static const _responseSchema = {
  'type': 'object',
  'properties': {
    'storeName': {'type': 'string'},
    'date':      {'type': 'string', 'nullable': true},
    'items': {
      'type': 'array',
      'items': {
        'type': 'object',
        'properties': {
          'name': {'type': 'string'},
          'price': {'type': 'integer'},
        },
        'required': ['name', 'price'],
      },
    },
  },
  'required': ['storeName', 'items'],
};
```

3-2. analyzeImage() — 메인 OCR 진입점

```
static Future<OcrResult?> analyzeImage(Uint8List imageBytes) async {
  final apiKey = dotenv.env['GEMINI_API_KEY'] ?? '';
  if (apiKey.isEmpty) return null;

  final base64Image = base64Encode(imageBytes); // 이미지 → Base64
  final url = 'https://generativelanguage.googleapis.com/v1beta/'
    'models/gemini-2.5-flash:generateContent?key=$apiKey';

  final body = {
    'contents': [{
```

```
'parts': [
  {'text': '이 영수증에서 상점명, 날짜, 모든 단품 상품(합계/결제금액 제외)을 추출해.'},
  {'inline_data': {'mime_type': 'image/jpeg', 'data': base64Image}},
],
}],
'generationConfig': {
  'temperature': 0, // 결정론적 출력
  'maxOutputTokens': 2048,
  'responseMimeType': 'application/json',
  'responseSchema': _responseSchema, // 구조 강제
},
};
// ... HTTP POST → JSON 파싱 → OcrResult 반환
}
```

**temperature: 0 설정 이유**

OCR은 창의적 응답이 필요 없고 영수증 내용을 정확히 옮겨야 합니다. temperature를 0으로 설정하면 동일 입력에 항상 동일한 결과를 보장합니다.

4

핵심 코드 리뷰 ② — Fallback 파싱 체계

lib/services/ocr_service.dart

4-1. ScanScreen 호출 흐름

①

Gemini 멀티모달 직접 파싱

Main Path

OcrService.analyzeImage(imageBytes) — 이미지 → OcrResult 직접 반환

②

Google Cloud Vision API

Fallback 1

analyzeImage 실패 시 → OcrService.extractText(imageBytes) → raw 텍스트 추출

③

Gemini 텍스트 파싱 → Regex

Fallback 2

raw 텍스트 → OcrService.parseReceiptText() → Gemini 텍스트 파싱 또는 Regex 3-pass

```
// ScanScreen._startOcr()
OcrResult? result = await OcrService.analyzeImage(_imageBytes!);

if (result == null) {                                // Fallback 진입
  final rawText = await OcrService.extractText(_imageBytes!);
  if (rawText == null || rawText.trim().isEmpty) {
    /* 에러 표시 */ return;
  }
  result = await OcrService.parseReceiptText(rawText);
}
```

4-2. 날짜 형식 정규화 — _parseFlexibleDate()

Gemini가 반환하는 날짜 형식이 영수증마다 달라서 `DateTime.tryParse()` 만으로는 실패합니다.

```
static DateTime? _parseFlexibleDate(String? raw) {
  if (raw == null) return null;

  // "2026.04.08" → "2026-04-08"
  var normalized = raw
    .replaceAll('년 ', '-').replaceAll('월 ', '-').replaceAll('일', '')
    .replaceAll('.', '-').replaceAll('/', '-').trim();
```



```
return DateTime.TryParse(normalized);  
// 처리 가능: "2026.04.08" / "2026/04/08" / "2026년 4월 8일"  
}
```



기존 Regex 파싱의 한계

영수증은 표 형태(가격이 상품명과 다른 줄에 있음)인 경우가 많아 "같은 줄에 가격 있음"을 전제한 Regex는 0개 검출 오류가 발생했습니다. Gemini는 레이아웃을 시각적으로 이해하므로 이 문제가 해결됩니다.

5

핵심 코드 리뷰 ③ — 웹 카메라 구현

lib/services/camera_picker_web.dart

5-1. 조건부 플랫폼 분기 (camera_picker.dart)

```
// Dart 조건부 export — 빌드 타임에 플랫폼별 파일 선택
export 'camera_picker_stub.dart'
    if (dart.library.js_interop) 'camera_picker_web.dart';
```

플랫폼	사용 파일	구현 방식
Flutter Web	camera_picker_web.dart	getUserMedia + HTML video + canvas 캡처
Android / iOS	camera_picker_stub.dart → ocr_service	ImagePicker(source: ImageSource.camera)

5-2. getUserMedia 카메라 오버레이 핵심 로직

```
Future<Uint8List?> pickFromCameraWeb() async {
  try {
    // 후면 카메라 스트림 요청
    final stream = await window.navigator.mediaDevices
      .getUserMedia(MediaStreamConstraints(
        video: MediaTrackConstraints(facingMode: 'environment'),
        audio: false,
      ));

    // 풀스크린 오버레이 생성
    final overlay = document.createElement('div') as HTMLDivElement
      ..style.position = 'fixed'
      ..style.inset = '0'
      ..style.zIndex = '9999'
      ..style.background = '#000';

    final video = document.createElement('video') as HTMLVideoElement
      ..srcObject = stream
      ..autoplay = true
      ..style.width = '100%'
```

```

    ..style.height = '100%'
    ..style.objectFit = 'cover';

    // 📷 촬영 버튼: canvas에 현재 프레임 캡처 → base64 → Uint8List
    captureBtn.onClick.listen((_) {
      final canvas = HTMLCanvasElement()
        ..width = video.videoWidth
        ..height = video.videoHeight;
      canvas.context2D.drawImage(video, 0, 0);
      final dataUrl = canvas.toDataURL('image/jpeg', 0.92);
      final bytes = base64Decode(dataUrl.split(',')[1]);
      completer.complete(bytes); // OCR로 전달
    });
  } catch (e) {
    return _fileFallback(); // 권한 거부 시 파일 선택기로 fallback
  }
}

```



dart:html 대신 package:web 사용 이유

Flutter 팀이 [dart:html](#) 을 deprecated 예정으로 공식 발표. [package:web](#) + [dart:js_interop](#) 이 공식 후속 패키지이며, 향후 Wasm 컴파일에도 호환됩니다.

6

기술적 의사결정 및 트레이드오프

6-1. OCR 엔진 선택: Gemini vs OpenAI vs Regex

방식	정확도	비용	레이턴시	결정
Regex 파싱만	낮음 (비정형 실패)	무료	즉시	Fallback 전용
Vision + Regex	중간 (표 형태 실패)	API 1회	1~2초	Fallback 2
OpenAI GPT-4o	높음	높음 (Input \$2.5/MTok)	2~4초	비용 과다
Gemini 2.5 Flash	높음	낮음 (무료 티어 포함)	1~2초	채택 ✓

6-2. 웹 카메라: getUserMedia vs image_picker

BEFORE — image_picker (웹) • 브라우저에서 파일 선택기(갤러리)만 열림 • 실시간 카메라 미리보기 불가 • 모바일 웹에서 후면 카메라 지정 불가 • 사진 촬영 UX 없음	AFTER — getUserMedia (웹) • 실시간 카메라 스트리밍 미리보기 • 후면 카메라 강제 지정 (facingMode: environment) • 촬영 버튼으로 즉시 캡처 • 권한 거부 시 파일 선택기로 안전하게 fallback
--	--

6-3. CORS 이슈 — 웹 환경의 제약

!

브라우저에서 외부 API 직접 호출 시 CORS 차단

네이버 쇼핑 API, OpenAI API 등은 브라우저 CORS 정책에 의해 웹에서 직접 호출이 차단됩니다. 현재는 개발 환경에서 `--disable-web-security` 플래그로 우회,

출시 전 서버 프록시 구현 예정

. Android/iOS 네이티브 앱은 CORS 제약 없이 정상 동작합니다.

7

Before / After 성능 비교

7-1. OCR 파싱 정확도

BEFORE — Regex 기반 파싱 - 매장명: "날짜" 잘못 추출 (2글자 한글 오탐) - 상품 인식: 0~2개 (표 형태 영수증 실패) - 날짜: "2026.04.08" 파싱 실패 (null) - 가격이 다른 줄에 있으면 미인식 - 비정형 영수증에서 완전 실패	AFTER — Gemini 멀티모달 - 매장명: 정확히 추출 (이마트 성수점) - 상품 인식: 5/5개 전량 추출 - 날짜: 다양한 형식 모두 정규화 성공 - 영수증 레이아웃 시각 이해로 표 형태도 처리 - 비정형 영수증에서도 안정적 동작
--	---

7-2. 테스트 영수증 파싱 결과 (이마트 성수점)

항목	Before (Regex)	After (Gemini)
매장명	날짜 (오탐)	이마트 성수점 ✓
날짜	null	2026-04-08 ✓
상품 수	0개	5/5개 ✓
총액	미인식	75,900원 ✓

7-3. 카메라 UX 개선

시나리오	Before	After
PC Chrome에서 카메라 버튼	파일 선택기 (갤러리) 열림	웹캠 실시간 미리보기 → 촬영
모바일 Chrome에서 카메라 버튼	파일 선택기 열림	후면 카메라 직접 실행
카메라 권한 거부	그냥 실패	파일 선택기로 안전하게 fallback

시나리오	Before	After
Android 네이티브 앱	갤러리 열림	카메라 앱 직접 실행

8 발표 흐름 가이드 — 데모 시연 순서

시연 순서

- 1

앱 홈 화면 보여주기

Buylog 앱 실행 — 기존 구매 이력이 Supabase에서 로드되는 모습
- 2

영수증 스캔 화면 이동

하단 탭 → 스캔 화면 이동. 카메라/갤러리 버튼 확인
- 3

카메라 버튼 → 실시간 미리보기

개선 포인트

카메라 버튼 클릭 → getUserMedia 오버레이 → 촬영 버튼으로 영수증 캡처
- 4

OCR 분석 → 결과 확인

개선 포인트

Gemini 2.5 Flash가 이미지를 분석 → 매장명, 날짜, 상품 목록이 자동 입력된 화면 표시
- 5

저장 → 홈 화면 반영 확인

저장 후 홈 화면에서 방금 등록한 상품이 즉시 반영되는 것 확인

Q&A 예상 질문

예상 질문	답변 포인트
Gemini API 비용이 걱정되지 않나요?	Gemini 2.5 Flash는 무료 티어(분당 15회, 일 1,500회)가 있어 학교 프로젝트 규모에서 비용 발생 없음. 유료 전환 시에도 GPT-4o 대비 약 10배 저렴.
영수증 종류마다 다 잘 되나요?	Gemini는 이미지를 시각적으로 이해하므로 표 형태, 열 배치, 다양한 폰트 모두 처리. Fallback 체계로 실패 시에도 안전하게 처리.
API 키가 앱에 포함되어 있으면 보안 문제 아닌가요?	현재는 개발 단계로 .env 파일에 저장하고 gitignore 처리. 출시 전 서버 프록시 구조로 전환 예정.
웹 카메라는 모바일에서도 되나요?	모바일 Chrome에서도 getUserMedia가 동작하며, facingMode: environment로 후면 카메라 직접 실행됨. 네이티브 앱은 ImagePicker 카메라 모드 사용.

 핵심 메시지

기존 Regex 기반 OCR의 한계를 Gemini 멀티모달로 극복해 비정형 영수증 파싱 정확도를 크게 향상시켰으며, Flutter Web의 카메라 제약을 Web API로 직접 해결해 자연스러운 촬영 경험을 구현했습니다.